LiDAR Point Cloud Reconstruction and Denoising Using DGCNN, KNN clustering and Motion-Aware Attention

Kshitij Gupta
Deep Snehalkumar Shah
Sabyrzhan Ilessov

MAE 598: Introduction to Autonomous Vehicles Engineering

Professor Ronald Calhoun

May 7th, 2026

Abstract

LiDAR sensors are widely used in autonomous driving because they provide accurate three-dimensional information about the surrounding environment. However, real LiDAR point clouds can be affected by noise, missing returns, and outlier points due to sensor limitations, object reflectance, distance, and adverse weather conditions. These corruptions can reduce the reliability of downstream autonomous-driving tasks such as perception, mapping, localization, and planning.

This project presents a chunk-based LiDAR point-cloud denoising and reconstruction system using KITTI Velodyne LiDAR data. Instead of processing an entire LiDAR frame at once, the point cloud is divided into local k-nearest-neighbor chunks . Artificial corruption is applied to clean chunks using Gaussian noise, point dropout, and random outliers. A Dynamic Graph Convolutional Neural Network (DGCNN) with motion-aware attention is then trained to reconstruct the clean point-cloud chunk from the corrupted input. The model also uses a corresponding previous-frame chunk to include temporal motion information.

The proposed network learns local geometric structure using EdgeConv layers and predicts a coordinate residual for each point. The final reconstruction is obtained by

adding this predicted residual to the corrupted input. The model is trained using a reconstruction loss that combines Chamfer Distance, pointwise L2 loss, repulsion loss, stability loss, and an attention-weighted error term. Experimental results on held-out KITTI sequence frames show that the model can reduce synthetic corruption and improve the local geometric quality of LiDAR point-cloud chunks. Although the system does not perform complete full-frame reconstruction, it demonstrates that chunk-based DGCNN reconstruction with motion-aware attention is a practical approach for local LiDAR denoising in autonomous-driving applications.

1 Introduction

Autonomous vehicles depend on accurate perception of the surrounding environment. Among the commonly used sensors, LiDAR is important because it directly measures three-dimensional structure in the form of point clouds. These point clouds support tasks such as object detection, mapping, localization, and motion planning.

However, LiDAR data collected in real driving conditions can contain noise, missing returns, and outlier points. Such corruption may come from sensor limitations, reflective surfaces, long-range measurements, object motion, or difficult weather conditions. If these errors are not handled properly, they can reduce the reliability hence the "SAFETY" of the autonomous-driving perception pipeline.

LiDAR reconstruction aims to recover a cleaner and more geometrically consistent point cloud from corrupted measurements. This is challenging because point clouds are sparse, unordered, and irregular, unlike images that are arranged on a fixed grid. Therefore, reconstruction methods must learn directly from local point geometry rather than relying on standard image-based convolution.

This project focuses on local LiDAR point-cloud reconstruction using a chunk-based deep learning approach. A Dynamic Graph Convolutional Neural Network (DGCNN) is used to learn geometric relationships between neighboring points, while a motion-aware attention module incorporates information from a previous LiDAR frame. The overall goal is to improve corrupted local point-cloud chunks while keeping the method computationally practical for autonomous-driving data.

2 Problem Statement

The problem addressed in this project is LiDAR point-cloud denoising and reconstruction. Given a corrupted local LiDAR point-cloud chunk, the goal is to predict a cleaner version that is close to the original clean chunk.

Let the clean point-cloud chunk be represented as

$$P = \{p_i\}_{i=1}^{N}, \qquad p_i \in \mathbb{R}^3, \tag{1}$$

where each point p_i contains the 3D coordinates (x, y, z) and $N = 2048$ is the number of points in one chunk. A corrupted version of this chunk is generated as

$$\tilde{P} = C(P), \tag{2}$$

where $C(\cdot)$ represents the artificial corruption process, including Gaussian noise, point dropout, and random outliers.

The model also uses a corresponding previous-frame chunk,

$$P_{\text{prev}} = \{p_i^{\text{prev}}\}_{i=1}^{N}, \tag{3}$$

to provide temporal motion information. Therefore, the reconstruction model can be written as

$$\hat{P} = f_\theta(\tilde{P}, P_{\text{prev}}), \tag{4}$$

where f_θ is the neural network with trainable parameters θ, and $\hat{P}$ is the reconstructed output chunk.

Instead of directly generating a new point cloud from scratch, the model predicts a coordinate residual ΔP for each corrupted point:

$$\Delta P = f_\theta(\tilde{P}, P_{\text{prev}}), \tag{5}$$

$$\hat{P} = \tilde{P} + \Delta P. \tag{6}$$

This residual formulation is useful because the corrupted input is already close to the desired clean structure in many regions. The network only needs to learn how to correct noisy, missing, or outlier-affected points.

The supervised learning objective is to minimize the difference between the reconstructed chunk $\hat{P}$ and the clean ground-truth chunk P:

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\hat{P}, P), \tag{7}$$

where $\mathcal{L}$ is the total reconstruction loss used during training. This loss combines geometric reconstruction accuracy, pointwise consistency, point distribution regularization, stability, and attention-weighted reconstruction error.

3 Dataset and Preprocessing

This project uses KITTI Velodyne LiDAR point-cloud data. Each LiDAR frame is stored as a binary file, where every point contains four values:

$$(x, y, z, r), \tag{8}$$

where x, y, and z represent the 3D point coordinates, and r represents the LiDAR intensity or reflectance value. In this project, only the spatial coordinates (x, y, z) are used, while the intensity value is ignored.

Each raw KITTI frame is loaded using NumPy and reshaped into a point-cloud matrix. The spatial point cloud is represented as

$$P_{\text{raw}} \in \mathbb{R}^{M \times 3}, \tag{9}$$

where M is the number of points in the original LiDAR frame. Since each full frame can contain a large number of points, direct full-frame reconstruction is computationally expensive. Therefore, the frame is divided into smaller local chunks before training.

3.1 Coordinate Normalization

Before chunking and training, the point coordinates are normalized by a fixed scale factor:

$$P_{\text{norm}} = \frac{P_{\text{raw}}}{50.0}. \tag{10}$$

This normalization keeps coordinate values in a smaller numerical range, which helps stabilize neural network training. The same normalization is applied to clean chunks, corrupted chunks, and previous-frame chunks.

3.2 Chunk Generation

The model is trained on local point-cloud chunks. Each chunk contains

$$N = 2048 \tag{11}$$

points. Chunk centers are selected from the full frame using farthest point sampling in the bird's-eye-view (x, y) plane. This helps distribute chunk centers across different regions of the scene.

For each selected center point, the nearest 2048 points are collected using k-nearest-neighbor search. A clean chunk is therefore defined as

$$P_j = \text{kNN}(P_{\text{norm}}, c_j, 2048), \tag{12}$$

where c_j is the selected chunk center and P_j is the resulting local point-cloud chunk.

A corresponding previous-frame chunk is also generated using the same center location. This previous-frame chunk provides temporal information for the motion-aware attention module.

3.3 Dataset Split

The main experiment uses sequence 00 from the KITTI Velodyne dataset. The dataset split used for the final training and testing setup is shown in Table 1.

Table 1: Dataset split used in the main experiment.

Set	**Frame Range**	**Number of Frames**
Training	000000–000050	51
Testing	000051–000065	15

For each frame, 30 local chunks are generated. Therefore, the training set contains

$$51 \times 30 = 1530 \tag{13}$$

training chunks per epoch, while the test set contains

$$15 \times 30 = 450 \tag{14}$$

test chunks.

3.4 Preprocessing Summary

The preprocessing pipeline can be summarized as follows:

1. Load a KITTI Velodyne `.bin` file.
2. Keep only the x, y, and z coordinates.
3. Normalize the coordinates by dividing by 50.0.
4. Select chunk centers using farthest point sampling in the BEV plane.
5. Extract 2048-point local chunks using k-nearest-neighbor search.
6. Generate matching previous-frame chunks for temporal input.

4 Corruption Model

To create supervised training pairs, artificial corruption is applied to each clean LiDAR chunk. The original clean chunk is used as the ground truth, while the corrupted chunk is used as the model input. For a clean chunk P, the corrupted chunk is written as

$$\tilde{P} = C(P), \tag{15}$$

where $C(\cdot)$ is the corruption function.

Three corruption types are used in this project. First, Gaussian noise is added to simulate LiDAR measurement error:

$$\tilde{p}_i = p_i + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2). \tag{16}$$

Second, point dropout is applied to simulate missing LiDAR returns. Third, random outlier points are added to simulate false or scattered measurements. The corruption is applied after chunk extraction, so each clean local chunk is corrupted independently.

The main corruption parameters used during training are shown in Table 2.

Table 2: Corruption parameters used during training.

Corruption Type	**Parameter**	**Value**
Gaussian noise	σ	0.008 normalized units
Point dropout	p_{drop}	0.15
Random outliers	N_{outlier}	30 points per 2048-point chunk

Because the point coordinates are normalized by dividing by 50.0, the Gaussian noise level of 0.008 corresponds to approximately 0.4 meters in the original coordinate scale. These corruptions allow the model to learn reconstruction behavior for noisy, incomplete, and outlier-contaminated point-cloud chunks.

5 Proposed Method and Model Architecture

The proposed method uses a chunk-based reconstruction pipeline. A clean KITTI LiDAR frame is first normalized and divided into local kNN chunks. Each clean chunk is then corrupted and used as the input to the reconstruction model. The model receives both the corrupted current-frame chunk and the corresponding previous-frame chunk. The previous-frame chunk is used by the motion-aware attention module to provide temporal information.

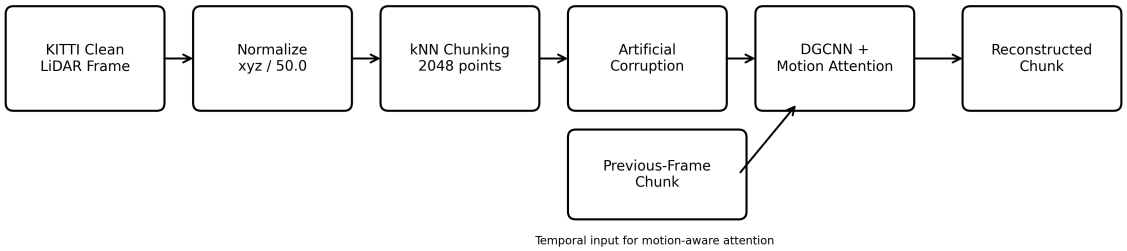

Figure 1: Overall LiDAR point-cloud reconstruction pipeline used in this project.

5.1 Model Input and Output

The reconstruction model takes two inputs:

$$\tilde{P} \in \mathbb{R}^{B\times 2048\times 3}, \tag{17}$$

$$P_{\text{prev}} \in \mathbb{R}^{B\times 2048\times 3}, \tag{18}$$

where $\tilde{P}$ is the corrupted current-frame chunk, P_{prev} is the previous-frame chunk, and B is the batch size. Each point contains only the normalized (x, y, z) coordinates.

The network predicts a 3D residual for each point:

$$\Delta P \in \mathbb{R}^{B\times 2048\times 3}. \tag{19}$$

The final reconstructed chunk is obtained by

$$\hat{P} = \tilde{P} + \Delta P. \tag{20}$$

This residual design allows the network to learn coordinate corrections instead of generating the full point cloud from scratch.

5.2 Motion-Aware Attention Module

The motion-aware attention module compares the current corrupted chunk with the previous-frame chunk. For each point, it uses four motion-related features:

$$d_i, \quad s_i, \quad v_{r,i}, \quad ttc_i, \tag{21}$$

where d_i is the distance from the sensor, s_i is the speed magnitude, $v_{r,i}$ is the radial motion, and ttc_i is an approximate time-to-collision proxy.

These four features are passed through a small multilayer perceptron:

$$4 \to 64 \to 64 \to 1. \tag{22}$$

A sigmoid activation produces one attention weight per point:

$$a_i \in [0, 1]. \tag{23}$$

The corrupted input is then gated using the attention weight:

$$x_{\text{gated},i} = x_i(1 + a_i). \tag{24}$$

This allows the model to emphasize points that may be more important for reconstruction.

5.3 DGCNN EdgeConv Encoder

The main encoder is based on Dynamic Graph CNN. For each point, a local graph is built using its k nearest neighbors, with

$$k = 16. \tag{25}$$

For each center point p_i and neighbor point p_j, the EdgeConv feature is formed as

$$e_{ij} = [p_j - p_i,\ p_i]. \tag{26}$$

This representation helps the network learn both relative local geometry and the original

point position.

The encoder contains three EdgeConv blocks. Their output feature dimensions are shown in Table 3.

Table 3: DGCNN EdgeConv encoder structure.

Block	**Input Channels**	**Output Channels**
EdgeConv 1	3	64
EdgeConv 2	64	128
EdgeConv 3	128	256

The outputs of the three EdgeConv blocks are concatenated:

$$64 + 128 + 256 = 448. \tag{27}$$

Therefore, each point has a 448-dimensional local geometric feature.

5.4 Global Feature and Decoder

A global feature is extracted by applying max pooling over all 2048 points:

$$\mathbb{R}^{B\times 2048\times 448} \to \mathbb{R}^{B\times 448}. \tag{28}$$

This global feature is passed through a global MLP:

$$448 \to 256 \to 256. \tag{29}$$

The resulting 256-dimensional global feature is repeated for every point and concatenated with the 448-dimensional local feature and the 3-dimensional gated input coordinate. Therefore, the decoder input size per point is

$$448 + 256 + 3 = 707. \tag{30}$$

The decoder MLP has the following structure:

$$707 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 3. \tag{31}$$

The final 3-dimensional output represents the predicted coordinate residual $(\Delta x, \Delta y, \Delta z)$ for each point.

5.5 Architecture Summary

The complete model can be summarized as a DGCNN-based point-cloud reconstruction network with motion-aware attention. The attention module uses previous-frame information to compute per-point importance weights. The EdgeConv encoder extracts local geometric features, the global MLP adds chunk-level context, and the decoder predicts a coordinate correction for each point.

The complete network contains 15 learnable Linear layers in total. For the reconstruction decoder alone, the input size is 707, the hidden layer sizes are 512, 256, and 128, and the output size is 3.

6 Loss Function

The model is trained using a combined reconstruction loss. The goal of this loss function is to make the reconstructed chunk $\hat{P}$ close to the clean ground-truth chunk P, while also encouraging stable and well-distributed point predictions.

The total loss is defined as

$$\mathcal{L}_{total} = \mathcal{L}_{CD} + 0.05\mathcal{L}_{L2} + 0.05\mathcal{L}_{rep} + 0.01\mathcal{L}_{stab} + 0.10\mathcal{L}_{attn}. \tag{32}$$

Here, $\mathcal{L}_{CD}$ is the Chamfer Distance, $\mathcal{L}_{L2}$ is the pointwise L2 loss, $\mathcal{L}_{rep}$ is the repulsion loss, $\mathcal{L}_{stab}$ is the stability loss, and $\mathcal{L}_{attn}$ is the attention-weighted reconstruction loss.

6.1 Chamfer Distance

Chamfer Distance is used as the main geometric reconstruction loss. It compares two unordered point sets by measuring nearest-neighbor distances between them:

$$\mathcal{L}_{CD}(\hat{P}, P) = \frac{1}{|\hat{P}|} \sum_{\hat{p} \in \hat{P}} \min_{p \in P} \|\hat{p} - p\|_2^2 + \frac{1}{|P|} \sum_{p \in P} \min_{\hat{p} \in \hat{P}} \|p - \hat{p}\|_2^2. \tag{33}$$

This term is important because point clouds do not have a fixed ordering. Therefore, nearest-neighbor comparison is more suitable than directly comparing points by index only.

6.2 Additional Loss Terms

The pointwise L2 loss encourages each predicted point to remain close to the corresponding clean point:

$$\mathcal{L}_{L2} = \frac{1}{N} \sum_{i=1}^{N} \|\hat{p}_i - p_i\|_2^2. \tag{34}$$

The repulsion loss discourages reconstructed points from collapsing too close together. This helps preserve a more realistic point distribution.

The stability loss limits unnecessary movement from the corrupted input:

$$\mathcal{L}_{stab} = \frac{1}{N} \sum_{i=1}^{N} \|\hat{p}_i - \tilde{p}_i\|_2^2. \tag{35}$$

This is useful because many corrupted points are already near their correct positions, so the model should only make corrections where needed.

The attention-weighted loss gives higher importance to points emphasized by the motion-aware attention module:

$$\mathcal{L}_{attn} = \frac{1}{N} \sum_{i=1}^{N} a_i \|\hat{p}_i - p_i\|_2^2, \tag{36}$$

where a_i is the attention weight for point i.

Together, these loss terms allow the model to learn geometric accuracy, local consistency, stable corrections, and attention-guided reconstruction.

7 Training Setup

The model was trained using supervised learning, where each training sample contains a corrupted LiDAR chunk, its clean ground-truth chunk, and a corresponding previous-frame chunk. During each training step, the corrupted and previous-frame chunks are passed into the network, the reconstructed chunk is predicted, and the total reconstruction loss is backpropagated to update the model parameters.

The main training configuration is shown in Table 4.

Table 4: Training configuration used in the main experiment.

Parameter	**Value**
Dataset	KITTI sequence 00
Training frames	000000–000050
Testing frames	000051–000065
Points per chunk	2048
Chunks per frame	30
DGCNN kNN size	16
Epochs	30
Batch size	1
Optimizer	Adam
Learning rate	1×10^{-4}
Coordinate normalization	Divide by 50.0
Corruption noise std.	0.008
Dropout probability	0.15
Random outliers	30 points per chunk

The model was trained on 51 frames from KITTI sequence 00. Since 30 chunks were extracted from each frame, each epoch contained

$$51 \times 30 = 1530 \tag{37}$$

training chunks. Testing was performed on 15 held-out frames from the same sequence, producing

$$15 \times 30 = 450 \tag{38}$$

test chunks.

The Adam optimizer was used with a learning rate of 1×10^{-4}. A batch size of 1 was used due to the memory and computational cost of graph construction, k-nearest-neighbor operations, and Chamfer Distance computation on 2048-point chunks.

During training, the model checkpoint and training history were saved for later evaluation. The recorded training history includes total loss, Chamfer Distance, L2 loss, repulsion loss, stability loss, and attention-weighted loss.

8 Evaluation and Results

The trained model was evaluated on 15 held-out frames from KITTI sequence 00, covering frames 000051 to 000065. These frames were not used during training. For each test frame, 30 local chunks were generated, giving a total of 450 test chunks.

The evaluation uses the same reconstruction loss components as training: total loss, Chamfer Distance, pointwise L2 loss, repulsion loss, stability loss, and attention-weighted loss. All reported metric values are computed in the normalized coordinate scale.

8.1 Training Performance

Figure 2 shows the training history for the main experiment. The total loss and Chamfer Distance decrease during training, which indicates that the model learns to reconstruct cleaner local point-cloud chunks from corrupted inputs.

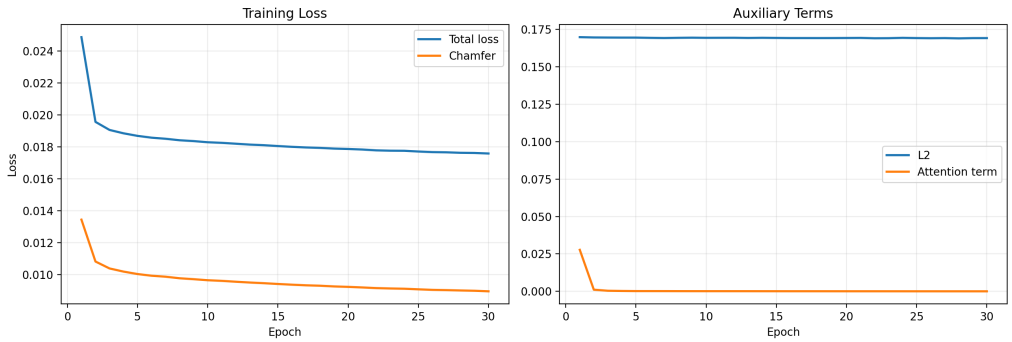

Figure 2: Training history for the DGCNN model trained on KITTI sequence 00.

At the final training epoch, the model achieved the results shown in Table 5.

Table 5: Final training results after 30 epochs.

Metric	Value
Total loss	0.01758
Chamfer Distance	0.00895
L2 loss	0.16916
Attention loss	0.000029

8.2 Held-Out Test Results

The final model was tested on 450 chunks from held-out frames. The average test results are shown in Table 6.

Table 6: Average test results on held-out KITTI sequence 00 frames.

Metric	Average Value
Total test loss	0.02033
Chamfer Distance	0.01202
L2 loss	0.16366
Repulsion loss	0.00103
Stability loss	0.00761
Attention loss	0.000042

The test loss is slightly higher than the final training loss, which is expected because the test frames were not seen during training. However, the results show that the model generalizes to nearby held-out frames from the same sequence and is able to reduce synthetic corruption in local LiDAR chunks.

8.3 Qualitative Visualization

Figure 3 shows a bird's-eye-view visualization from a held-out test frame. The BEV view is useful because it clearly shows the top-down geometric structure of the LiDAR scene. In the visualization, the reconstructed result is compared with the corrupted input and the clean ground-truth chunk.

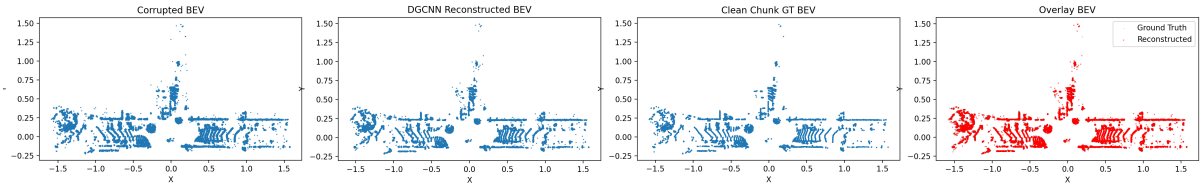

Figure 3: Bird's-eye-view comparison of corrupted input, reconstructed output, and clean ground truth for a held-out KITTI test frame.

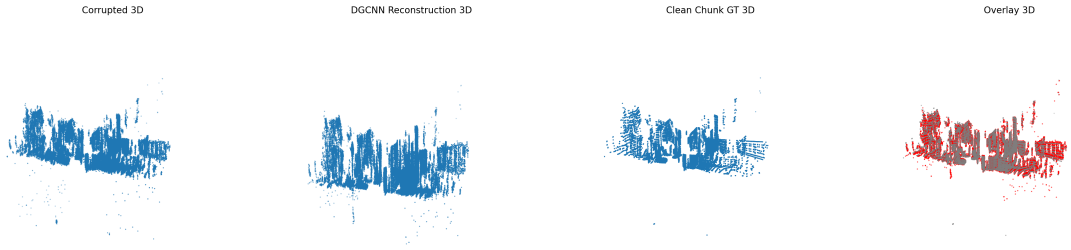

Figure 4: 3D visualization of the corrupted input, reconstructed output, and clean ground truth for a held-out KITTI test frame.

The qualitative result shows that the reconstructed point cloud is generally cleaner than the corrupted input. The model reduces the effect of random outliers and helps preserve the local geometric shape of the original LiDAR chunk. However, because chunks are reconstructed independently, the method should be interpreted as local point-cloud denoising rather than complete full-frame LiDAR reconstruction.

9 Discussion

The results show that the model can reduce synthetic corruption in local KITTI LiDAR chunks. The decrease in training loss shows that the network learned to map corrupted chunks toward their clean ground-truth versions. The held-out test results also show reasonable performance on unseen frames from the same sequence.

The main strength of the method is its local geometric learning. DGCNN EdgeConv layers are suitable for point clouds because they learn relationships between neighboring points instead of assuming a fixed grid structure. This helps the model preserve local shape while reducing Gaussian noise and outlier effects.

The residual output design also supports stable reconstruction. Instead of predicting a

completely new point cloud, the model predicts a coordinate correction for each input point:

$$\hat{P} = \tilde{P} + \Delta P. \tag{39}$$

This is useful because many corrupted points are still close to the clean geometry, so the network only needs to learn how to adjust them.

The motion-aware attention module provides additional temporal context from the previous LiDAR frame. This helps the network assign higher importance to points that may be more relevant for reconstruction due to motion or distance-based effects.

Overall, the method works best as a local chunk-level denoising and reconstruction approach. It improves corrupted point-cloud chunks, but it should not be interpreted as a complete full-frame LiDAR reconstruction system because the chunks are processed independently.

10 Limitations

Although the proposed method shows promising local reconstruction results, it has several limitations.

First, the model is trained and tested only on KITTI sequence 00. The test frames are held out from training, but they still come from the same driving sequence. Therefore, cross-sequence and cross-dataset generalization were not fully evaluated.

Second, the corruption used in this project is synthetic. Gaussian noise, point dropout, and random outliers are useful for controlled training, but they do not fully represent complex real-world LiDAR degradation caused by rain, fog, snow, reflective surfaces, or sensor physics.

Third, the method performs chunk-level reconstruction rather than full-frame reconstruction. Each 2048-point chunk is reconstructed independently, so the system does not currently perform global merging or enforce consistency between overlapping chunks.

Finally, only the (x, y, z) coordinates are used. The LiDAR intensity value and semantic object labels are not included. This limits the model to geometric reconstruction without object-level or material-level information.

11 Future Work

Several improvements can be explored in future work. First, the reconstruction pipeline can be extended from independent chunk-level reconstruction to full-frame reconstruction. This would require a better merging strategy for overlapping reconstructed chunks, such as averaging nearby predictions or removing duplicate points.

Second, more realistic LiDAR corruption models can be added. Instead of only using Gaussian noise, dropout, and random outliers, future experiments can include weather-based corruption such as rain, fog, snow, distance-dependent noise, and beam dropout.

Third, the model can be trained and evaluated on more KITTI sequences. This would help test whether the method generalizes to different driving environments, road layouts, and traffic conditions.

Fourth, additional input features can be used. The current model only uses (x, y, z) coordinates, but future work could include LiDAR intensity or semantic labels if available. This may help the model understand object boundaries and surface properties more effectively.

Finally , a future improvement is to extend our model into a **hierarchical KNN-DGCNN framework** to better preserve global scene structure. Instead of reconstructing only independent local chunks, the full LiDAR frame could first be divided into small point-level chunks for detailed local reconstruction. These chunks could then be treated as higher-level cluster nodes, where a second KNN graph models relationships between neighboring chunks. This would allow local point interactions to preserve fine geometry while chunk-level interactions provide broader scene context. Such a hierarchy could improve global consistency without requiring expensive full-scene graph processing.

12 Conclusion

This project presented a chunk-based LiDAR point-cloud reconstruction and denoising method for autonomous-driving data. The system uses KITTI Velodyne LiDAR frames, extracts local 2048-point chunks, applies artificial corruption, and trains a DGCNN-based model to reconstruct cleaner point-cloud chunks.

The proposed model combines EdgeConv-based local geometric learning with motion-aware attention from a previous LiDAR frame. Instead of directly generating a new point cloud, the network predicts a coordinate residual that is added to the corrupted input.

This makes the reconstruction task more stable and suitable for denoising.

The model was trained on KITTI sequence 00 and evaluated on held-out frames from the same sequence. The results show that the method can reduce synthetic noise and outlier effects while preserving local geometric structure. However, the method is still limited to local chunk-level reconstruction and does not perform complete full-frame reconstruction.

Overall, this project demonstrates that chunk-based DGCNN reconstruction with motion-aware attention is a practical and computationally feasible approach for local LiDAR point-cloud denoising in autonomous-driving applications.

A Appendix: Code and Dataset Access

The complete implementation for this project, including the DGCNN reconstruction model, kNN-based chunk generation, corruption pipeline, motion-aware attention module, training scripts, testing scripts, and visualization utilities, is available in the project code repository:

`github link`

The LiDAR data used in this project is based on the KITTI Vision Benchmark Suite. The official KITTI dataset can be accessed from:

`https://www.cvlibs.net/datasets/kitti/`

For this project, KITTI Velodyne LiDAR sequence 00 was used for the main training and testing experiments. The model was trained on frames 000000–000050 and evaluated on held-out frames 000051–000065.

A.1 Repository Contents

The code repository contains the following main components:

- Dataset loading and preprocessing scripts for KITTI Velodyne point clouds.
- kNN-based local chunk extraction from full LiDAR frames.
- Synthetic corruption module using Gaussian noise, point dropout, and random outlier injection.

- Motion-aware attention module using previous-frame LiDAR information.
- DGCNN-based point-cloud reconstruction model.
- Training and evaluation scripts.
- BEV and 3D visualization scripts for qualitative result analysis.

A.2 Reproducibility Notes

To reproduce the experiments, the KITTI dataset should be downloaded separately from the official KITTI website and placed in the expected directory structure described in the repository documentation. The trained model checkpoints and generated visualizations are saved under the project `experiments/` directory.

References

[1] A. Geiger, P. Lenz, and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3354–3361.

[2] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic Graph CNN for Learning on Point Clouds," *ACM Transactions on Graphics*, vol. 38, no. 5, 2019.

[3] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 652–660.

[4] H. Fan, H. Su, and L. J. Guibas, "A Point Set Generation Network for 3D Object Reconstruction from a Single Image," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 605–613.

[5] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[6] M.-J. Rakotosaona, V. La Barbera, P. Guerrero, N. J. Mitra, and M. Ovsjanikov, "PointCleanNet: Learning to Denoise and Remove Outliers from Dense Point Clouds," *Computer Graphics Forum*, vol. 39, no. 1, pp. 185–203, 2020.

[7] A. Hahner, C. Sakaridis, D. Dai, and L. Van Gool, "Fog Simulation on Real LiDAR Point Clouds for 3D Object Detection in Adverse Weather," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.